Controlador FPGA para transceptor LoRa SX1278

Interfaz UART, controlador SPI y gestión de CONFIG, TX y RX

Circuitos Lógicos Programables

Trabajo práctico final

Plataforma: Zybo Zynq-7000

Periférico: SX1278, 433 MHz

4to bimestre de 2026

Índice

1. Introducción **1**

1.1. Funciones implementadas 1

1.2. Correspondencia con el enunciado 1

2. Arquitectura del circuito **2**

2.1. Flujo general 2

2.2. Interfaz UART y decoder de comandos 3

3. Controlador SX1278 **3**

3.1. Manager de configuración 3

3.2. Manager de transmisión 4

3.3. Manager de recepción 4

4. Cadena SPI **5**

5. Simulación y validación **6**

5.1. Simulaciones realizadas 6

5.2. Validación sobre hardware 7

5.3. Síntesis, implementación y configuración de la FPGA 9

6. Uso de recursos de la FPGA **10**

6.1. Resultado temporal 10

7. Conclusiones **10**

1. Introducción

El trabajo implementa en VHDL un controlador para el transceptor LoRa SX1278 sobre una placa Zybo Zynq-7000. El sistema recibe órdenes desde una PC mediante UART, las interpreta y genera las transacciones SPI necesarias para configurar el transceptor, transmitir payloads y recuperar paquetes recibidos.

La naturaleza del producto es la de un **módem LoRa serie**: el equipo host intercambia comandos y datos por UART, mientras el controlador oculta los registros, las interrupciones y la temporización del SX1278. Este enfoque se utiliza habitualmente en módulos de comunicación controlados mediante una interfaz serie; por ejemplo, en dispositivos como el ESP8266 cuando opera como módem. En este proyecto se definió un protocolo binario propio para CONFIG, TX y RX.

El diseño utiliza un reloj de sistema de **125 MHz**, una UART **115200 8N1** y un bus SPI configurable. Para el alcance del trabajo práctico se adoptó un payload máximo de **50 bytes**.

1.1. Funciones implementadas

- Configuración de frecuencia, ancho de banda, coding rate, spreading factor, preámbulo y potencia de transmisión.
- Aplicación de una secuencia completa de configuración sobre SPI.
- Reset físico del SX1278.
- Carga byte a byte del payload y escritura de la FIFO TX.
- Inicio de transmisión y detección de `TxDone` mediante DIO0.
- Inicio de recepción y lectura automática de IRQ, longitud y FIFO RX.
- Envío automático del paquete recibido hacia la PC mediante UART.
- Respuestas UART con ACK/NACK y checksum XOR.
- Indicadores LED de actividad, ocupación, finalización y error.

1.2. Correspondencia con el enunciado

El desarrollo se realizó íntegramente en VHDL y se implementó sobre el kit Zybo Zynq-7000. La Tabla 1 resume la evidencia incluida en este documento para los requisitos técnicos del trabajo final.

Cuadro 1: Correspondencia entre los requisitos y la implementación.

Requisito	Evidencia presentada
Bloque digital en VHDL	Controlador UART–SPI para el SX1278, organizado en componentes sintetizables.
Simulación de componentes	Testbenches VHDL y entorno automatizado para UART, SPI, decoder y controlador SX1278.
Simulación del bloque principal	Testbench de integración de `lora_command_fpga_manager`.
Síntesis e implementación	Reportes posteriores a placement y ruteo, con cierre temporal a 125 MHz.
Archivo de configuración	Bitstream `lora_command_fpga_manager.bit` generado por Vivado.
Prueba sobre FPGA	Capturas del bus SPI y funcionamiento con el SX1278 conectado a la Zybo.

2. Arquitectura del circuito

El top level `lora_command_fpga_manager` se limita a interconectar los subsistemas. La lógica se dividió en bloques pequeños para separar el protocolo UART, el control del SX1278 y la generación de señales físicas SPI.

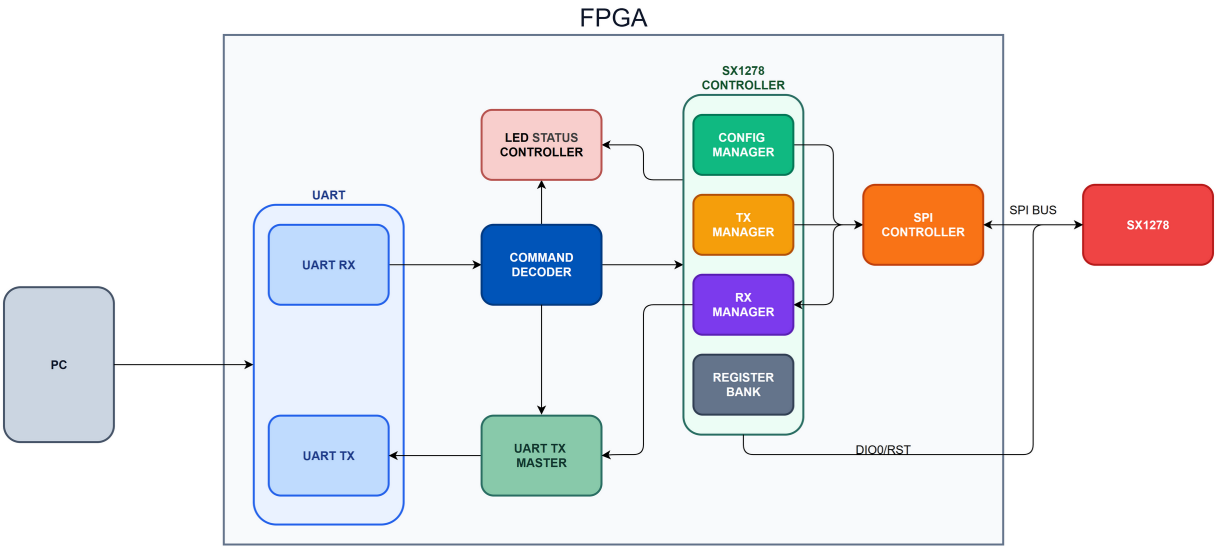


Figura 1: Diagrama en bloques del sistema completo.

2.1. Flujo general

1. La UART reconstruye los bytes recibidos desde la PC.
2. `command_decoder` recibe una trama completa y verifica delimitadores, checksum, comando y parámetros.
3. Si la orden es válida se genera un pulso hacia `sx1278_controller`; de lo contrario se responde NACK.
4. El controlador selecciona el manager de CONFIG, TX o RX.
5. El manager solicita una secuencia al controlador SPI compartido.
6. La cadena SPI convierte cada frame en `NSS`, `SCLK`, `MOSI` y captura `MISO`.
7. Las respuestas y los paquetes recibidos se serializan por UART.

2.2. Interfaz UART y decoder de comandos

La interfaz UART utiliza una trama de longitud fija:

```
[#] [COMANDO] [PARAMETRO] [VALOR] [CHECKSUM] [$]
```

El checksum es el XOR de comando, parámetro y valor. El decoder espera la trama completa antes de ejecutar una acción. Ante checksum, delimitador o comando incorrecto responde NACK, permitiendo que el host reintente.

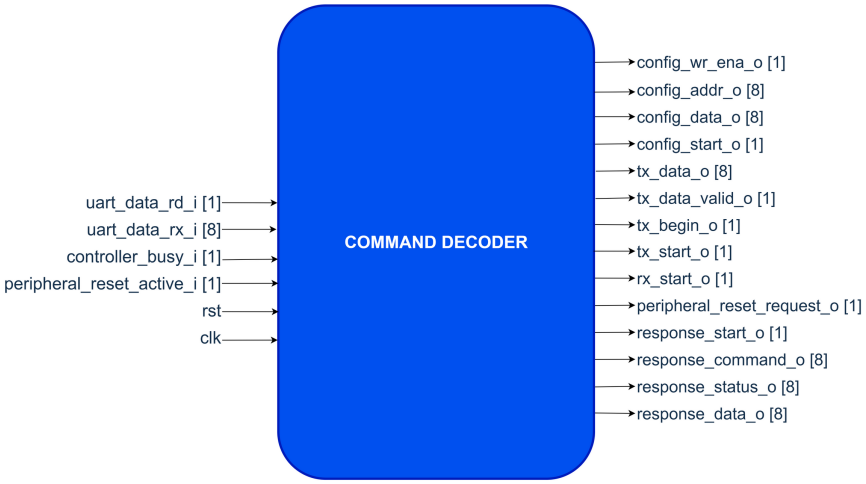


Figura 2: Interfaz del decoder de comandos UART.

3. Controlador SX1278

El controlador arbitra tres managers independientes. Todos comparten el mismo secuenciador SPI y sólo uno puede solicitarlo en cada operación. La señal DIO0, asíncrona respecto de `clk`, atraviesa un sincronizador y un detector de flanco antes de ingresar a las máquinas de estado.

3.1. Manager de configuración

`sx1278_config_manager` recorre la secuencia de registros necesaria para llevar el radio a LoRa, cargar los parámetros configurables, posicionar las bases de FIFO y limpiar las interrupciones.

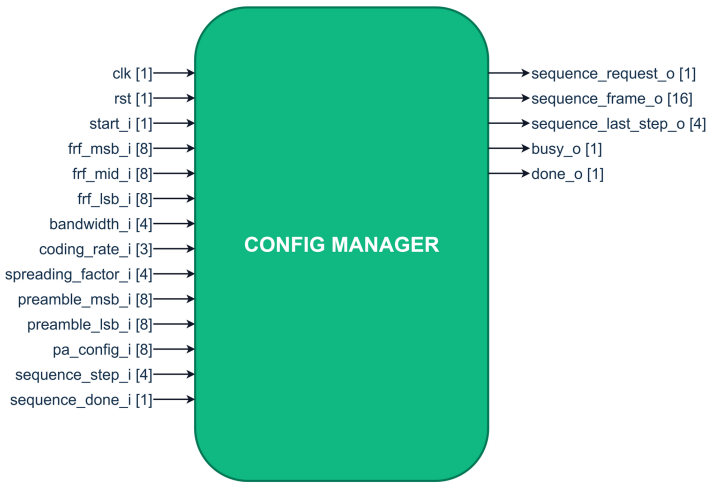


Figura 3: Entradas y salidas del manager de configuración.

3.2. Manager de transmisión

`sx1278_tx_manager` mantiene un buffer local de hasta 50 bytes. Al recibir la orden de transmisión prepara el puntero de FIFO, carga el payload, programa la longitud, configura DIO0 como `TxDone`, inicia TX y espera el flanco de DIO0.

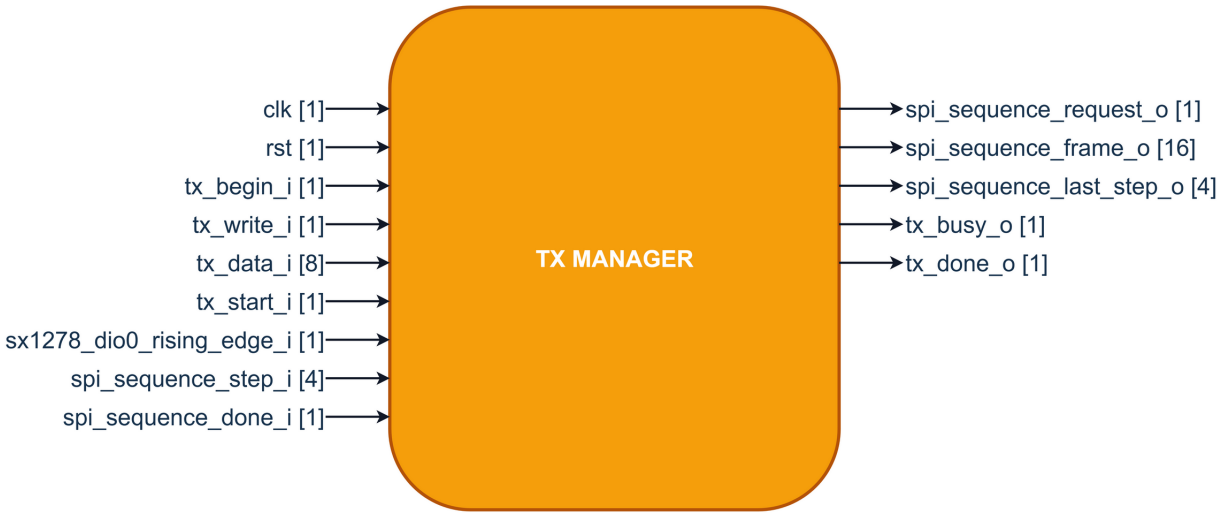


Figura 4: Entradas y salidas del manager de transmisión.

3.3. Manager de recepción

`sx1278_rx_manager` inicia el modo de recepción y espera DIO0. Luego lee los flags IRQ, descarta paquetes con CRC incorrecto, obtiene longitud y dirección de FIFO, almacena el payload y lo presenta al serializador UART.

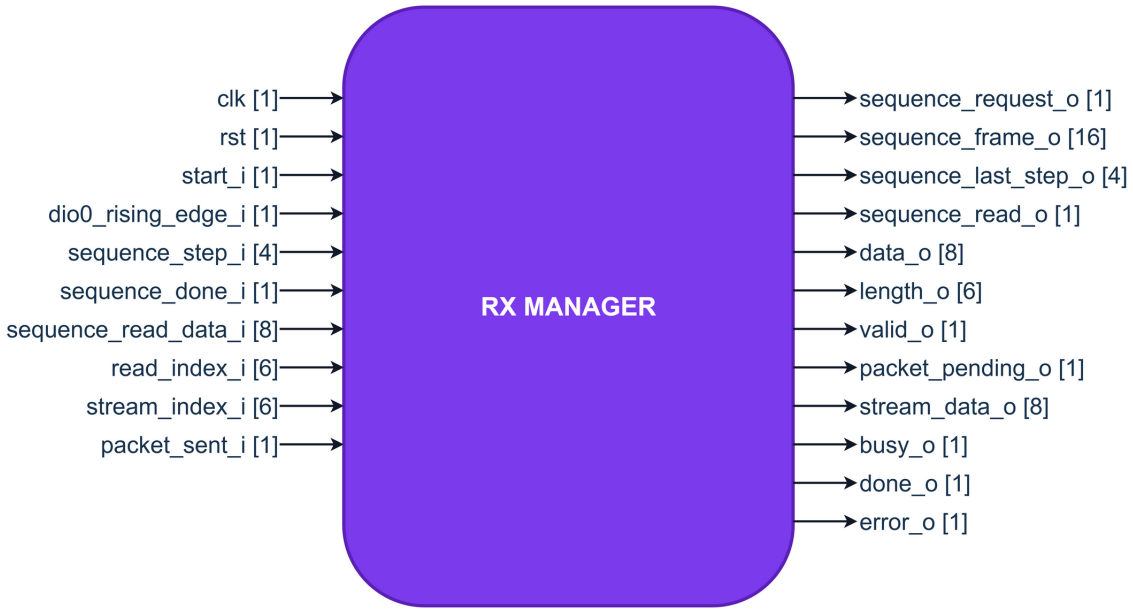


Figura 5: Entradas y salidas del manager de recepción.

4. Cadena SPI

La cadena SPI se divide en tres niveles:

- `spi_sequence_controller`: ejecuta una lista de accesos a registros.
- `spi_register_access`: adapta una lectura o escritura de registro a dos bytes SPI.
- `spi_frame_controller` y `spi_pin_driver`: almacenan los bytes, mantienen NSS durante el frame y generan o capturan las señales físicas.

Esta separación evita que las MEF de CONFIG, TX y RX tengan que generar flancos de reloj o administrar directamente los bytes del bus.

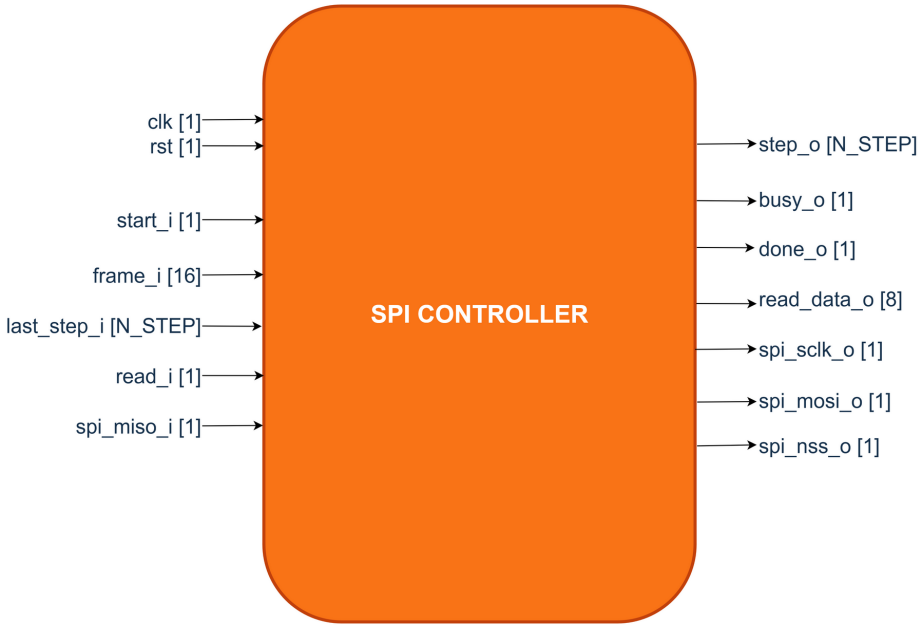


Figura 6: Interfaz del controlador de secuencias SPI.

5. Simulación y validación

5.1. Simulaciones realizadas

La verificación se realizó en dos niveles. Los testbenches de componente comprueban las interfaces básicas y el testbench del top comprueba el flujo completo. De esta forma, los errores del protocolo UART o del bus SPI pueden aislarse antes de ejecutar la integración.

Cuadro 2: Niveles de simulación utilizados.

Bloque verificado	Prueba principal
UART TX y UART RX	Transmisión y recepción de bytes individuales y secuencias de bytes a 115200 baud.
SPI pin driver	Modos de polaridad y fase, datos `A5` y `81`, temporización de SCLK y muestreo de MISO.
SPI frame controller	Carga de varios bytes, control de NSS, pausa entre bytes y captura de la respuesta.
Decoder de comandos	Delimitadores, checksum XOR, ACK/NACK y generación de strobes de CONFIG, TX y RX.
Controlador SX1278	Secuencias de configuración, carga de TX, espera de DIO0 y recuperación del payload RX.
Top completo	Comandos UART, actividad SPI, transmisión de payloads y devolución de un paquete recibido.

El testbench integrado `lora_command_fpga_manager_tb.vhd` ejercita el camino completo UART–decoder–controlador–SPI. Las pruebas principales son:

- rechazo de checksum y delimitadores UART incorrectos;
- escritura y aplicación de configuración;
- transmisión de payloads mediante el SX1278;
- recepción y envío automático de los datos por UART.

Una ejecución correcta finaliza con:

```
UART verificada: CONFIG, TX y RX automatico correctos
```

La Figura 7 presenta una ejecución del testbench de integración. Se observan la actividad UART del host, las respuestas generadas por la FPGA, el estado indicado por los LED y el comienzo de la actividad SPI.

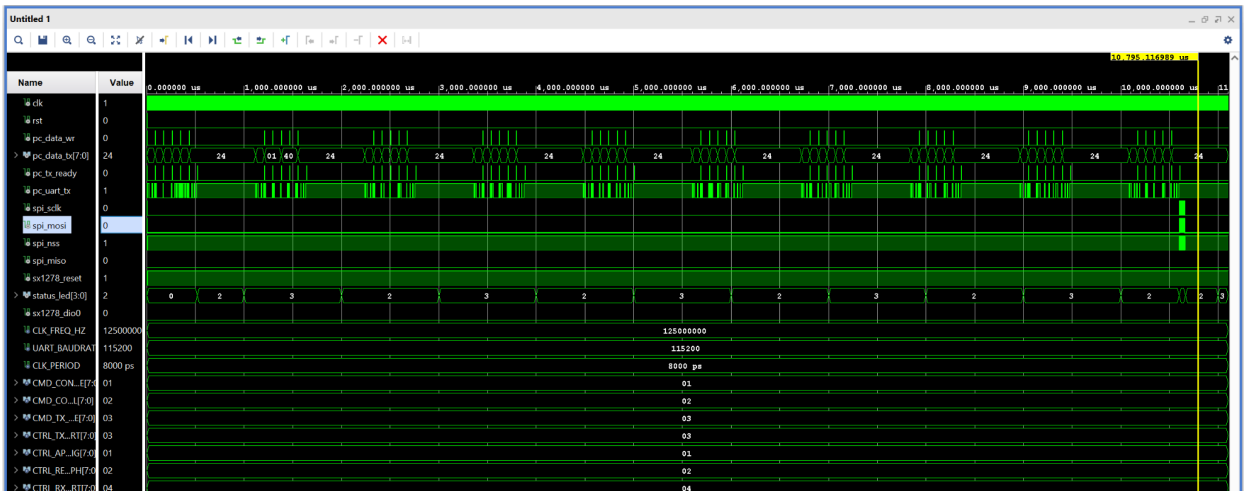


Figura 7: Simulación del top completo en Vivado.

5.2. Validación sobre hardware

El diseño se implementó sobre la FPGA Zynq y se conectó a un transceptor SX1278. Como nodo LoRa complementario se utilizó una placa Heltec ESP32 LoRa, mientras que un analizador lógico permitió observar las interfaces digitales durante las pruebas. El montaje experimental se muestra en la Figura 8.

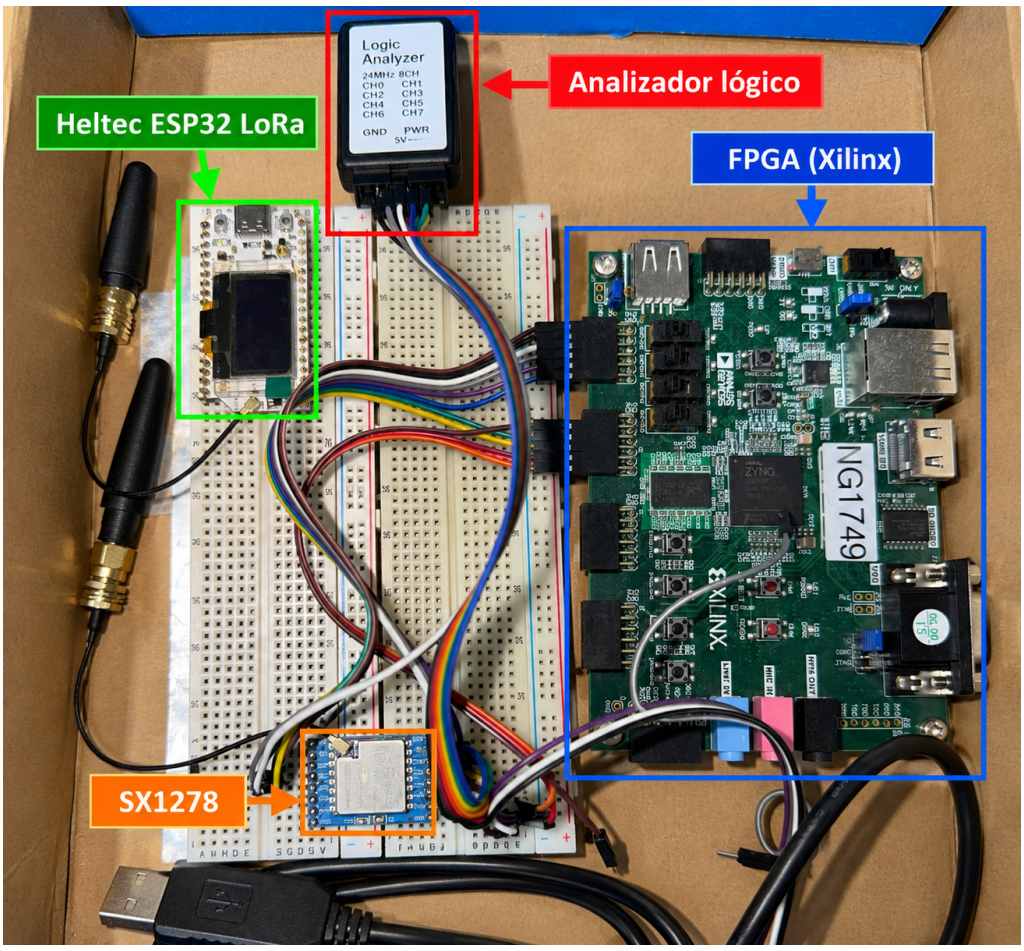


Figura 8: Montaje experimental empleado para validar el controlador LoRa implementado en la FPGA.

Además de verificar el intercambio LoRa, se observó el bus físico mediante el analizador lógico. La Figura 9 muestra la secuencia SPI de configuración y la Figura 10 el inicio del modo de recepción.

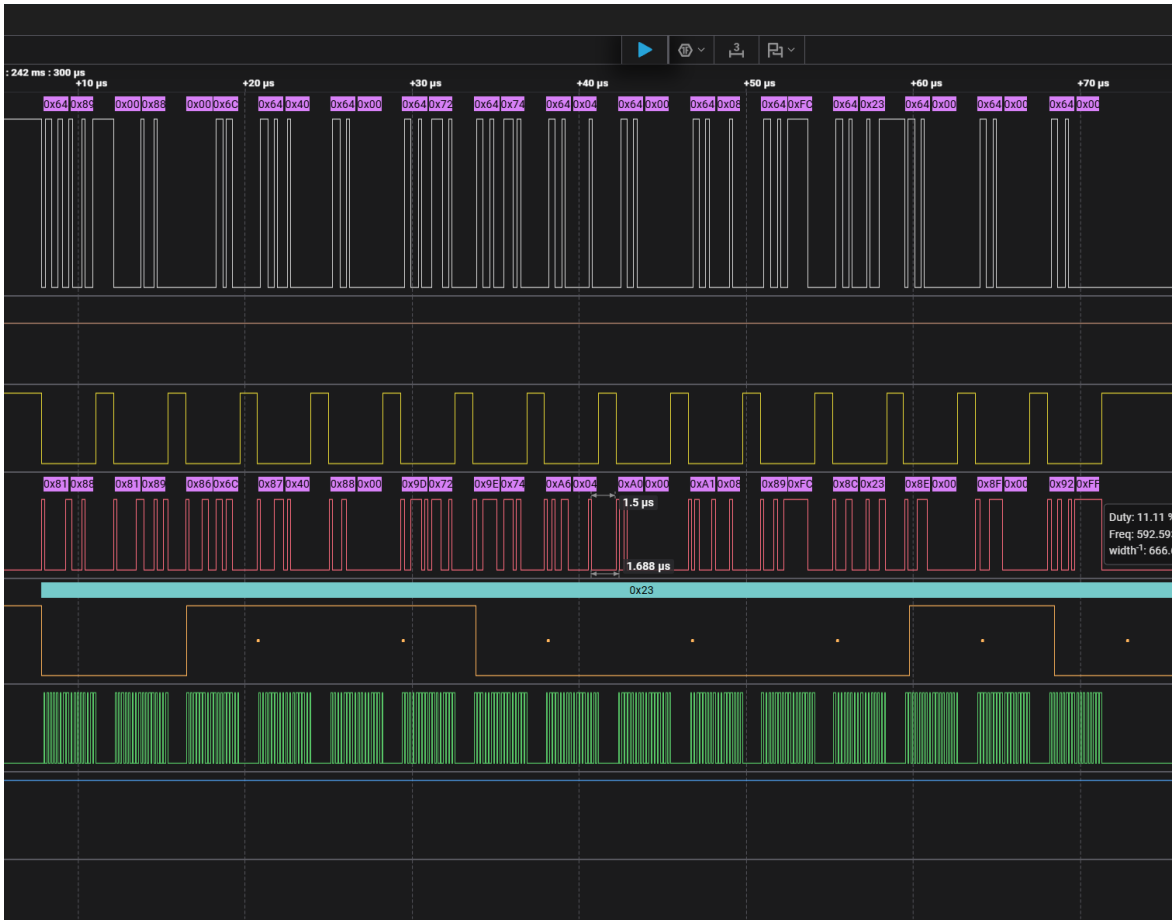


Figura 9: Secuencia SPI de configuración observada en hardware.

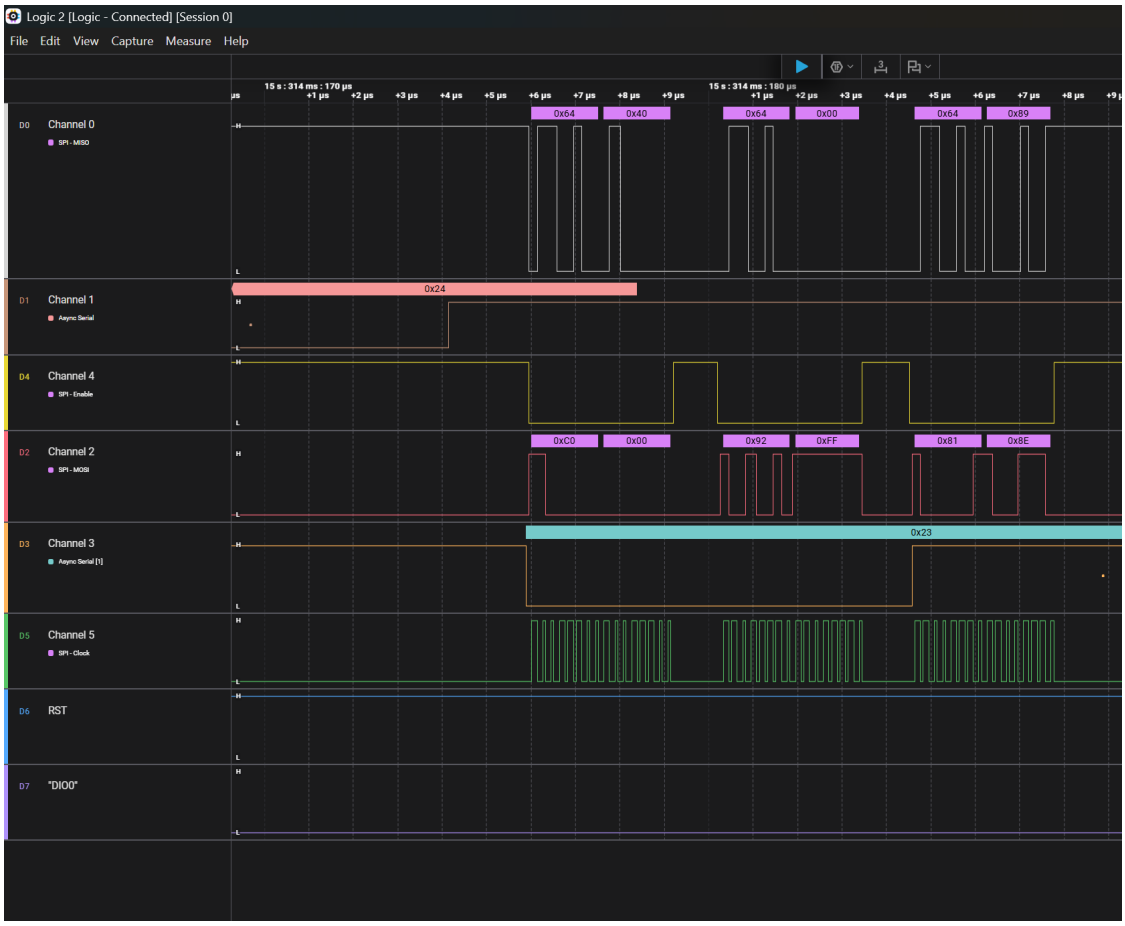


Figura 10: Configuración de DIO0, limpieza de IRQ e ingreso al modo RX.

Durante la integración se utilizó una separación de **20 ms** entre comandos UART y hasta tres reintentos. Esta configuración permitió completar transmisiones consecutivas sin superponer órdenes mientras el controlador se encontraba ocupado.

5.3. Síntesis, implementación y configuración de la FPGA

Vivado sintetizó e implementó el top `lora_command_fpga_manager`. El flujo completó placement y ruteo, generó los reportes de utilización y timing, y produjo el archivo de configuración:

```
LORA_FPGA.runs/impl_1/lora_command_fpga_manager.bit
```

El bitstream se descargó en la Zybo y el funcionamiento se comprobó con un adaptador UART conectado a la PC, el módulo SX1278 y un analizador lógico. Las capturas anteriores muestran las transacciones SPI obtenidas durante esa validación.

6. Uso de recursos de la FPGA

La Tabla 3 se obtuvo del reporte de utilización posterior a placement de Vivado para el top `lora_command_fpga_manager`.

Cuadro 3: Utilización de recursos de la Zybo Zynq-7000.

Recurso	Utilizado	Disponible	Utilización
Slice LUTs	851	17 600	4,84 %
Slice Registers	1 333	35 200	3,79 %
F7 Muxes	89	8 800	1,01 %
F8 Muxes	10	4 400	0,23 %
Block RAM Tile	0	60	0,00 %
DSP	0	80	0,00 %
Bonded IOB	14	100	14,00 %
BUFGCTRL	1	32	3,13 %

El diseño utiliza principalmente lógica distribuida y flip-flops. Las memorias pequeñas destinadas a TX, RX y al frame SPI fueron implementadas por síntesis sin consumir bloques BRAM dedicados.

6.1. Resultado temporal

Cuadro 4: Resumen temporal posterior al ruteo.

Parámetro	Resultado
Frecuencia del reloj	125 MHz
Período	8,000 ns
WNS	0,280 ns
TNS	0,000 ns
WHS	0,139 ns
THS	0,000 ns

Vivado informa que **todas las restricciones temporales definidas por el usuario se cumplen**. Los datos completos se encuentran en los reportes `lora_command_fpga_manager_utilization_placed.rpt` y `lora_command_fpga_manager_timing_summary_routed.rpt`.

7. Conclusiones

El trabajo implementa una cadena completa de control LoRa en lógica programable: recibe comandos de una PC, valida el protocolo, administra las operaciones del SX1278 y genera el bus SPI. La división entre decoder, managers y controladores físicos permite verificar cada módulo de manera aislada y mantener las máquinas de estado enfocadas en una única responsabilidad.

La implementación ocupa menos del 5 % de las LUT y registros disponibles y cumple timing a 125 MHz, dejando margen para incorporar nuevas funciones sin cambiar la arquitectura principal.

Los secuenciadores dedicados fueron adecuados para el trabajo, aunque no siempre son la alternativa más eficiente. Una evolución podría concentrar las secuencias repetitivas en un *IP core* AXI y delegar al procesador del SoC Zynq la configuración y el control de alto nivel, simplificando las máquinas de estado.